上位机界面优化需求（S5 之后 · 界面交互批次）

- **提出**：用户于自查完成后提出 6 条界面优化/修复需求（`m06024`），本文件是沟通整理稿
- **整理日期**：2026-10-13
- **范围**：`host/` 的界面层（`ui_kit.py`、`main_window.py`、`panel_wave.py`、`panel_abz.py`、`panel_record.py`、`panel_*.py`、`theme.py`），**不动**协议层 `link.py`、解析层 `encoder.py` / `abz.py` / `signal.py` / `regs.py` 的对外语义
- **硬约束**：不新增第三方依赖（只用现有 PySide6 + matplotlib + 标准库）；界面文案继续全中文；离线自检（现 101 项 + 界面级 30 项）与真机 S1–S5 必须保持全绿；`export_png.py` 的 1950×1080 / 1950×990 像素契约不变

R1 全界面滚轮顺畅滑动（✅ 批次 1 已实现）

现状与原因 - 主窗口已有 `QScrollArea`（`host/main_window.py:399-408`，`setWidgetResizable(True)` + 竖向滚动条常显），所以问题不是"没做滚动"； - 滚轮事件被卡片内的子控件吃掉：matplotlib 画布（`FigureCanvasQTAgg`）、端口表、下拉框、数字框各自处理滚轮，鼠标停在上面时页面不动； - 默认按行/段滚动、卡片高度大，观感"一跳一跳"。

已确认做法 1. 主窗口安装**应用级事件过滤器**：除少数明确例外，所有 `QEvent.Wheel` 一律转成滚动条的**像素级滚动**（`setSingleStep` 调小 + `QScroller.grabGesture` 或手动 `setValue`），实现平滑跟随。 2. **数字框（`QSpinBox` / `QDoubleSpinBox`）与下拉框（`QComboBox`）不再吃滚轮**（置 `setFocusPolicy` + 屏蔽 wheel 事件），改值改用键盘输入/上下箭头或点击展开 —— 避免鼠标扫过误改参数。 3. 图例/表格等需要保留自身滚动的控件（如日志框 `QPlainTextEdit`、端口表格）**列入白名单**：鼠标在其上时滚它们自己，不劫持。 4. 不加惯性（Qt 无内置惯性，且用户未选该项）。

验收：`selftest_ui.py` 新增用例——构造滚轮事件派发到卡片、画布、下拉框、日志框上，断言"页面滚动条值发生变化"（白名单控件断言"自身滚动、页面不动"）。

R2 输入信号波形：示波器式可调（✅ 批次 3 已实现）

现状：`host/panel_wave.py:207-210` 横轴写死 `-capacity/10.0 … 0.0`（ms）、纵轴写死 `±lim`，除"暂停刷新"外无任何视图控制。

已确认做法 - **默认自动跟随信号**：按最新 `f_in` 显示 1~2 个输入周期（横轴时间窗自动取整到 1/2/5/10/20/50/100 ms 档），**手动一调即锁定**，**双击图内恢复自动**。 - 横轴：时间窗档位下拉 + 拖动平移（鼠标左键拖 = 平移，滚轮不动图，见 R1）。 - 纵轴：每格幅值档位（Q1.15 满量程比例：1.0 / 0.5 / 0.25 / 0.1）+ 垂直偏移 + "自动量程"按钮。 - 李萨如子图同步缩放并保持等比例（`set_aspect("equal")` 不变）。 - 标注补全：`横轴 = 时间 (ms)`（0 = 最新帧）、`纵轴 = 幅值 (Q1.15，满量程 ±1.0 ≡ ±32768 LSB)`、网格刻度数值、图例（`s_n` / `c_n`）、当前视图范围读数。 - 游标测量：两条可拖竖线显示 `Δt (ms / 输入周期数)` 与 `ΔV (LSB / Q1.15)`（可选开关）。

验收：`selftest_ui.py` 断言默认自动量程随 `f_in` 变化、手动设定后不再被覆盖、双击恢复自动；离线断言轴标签/单位文本存在。

R3 ABZ/UVW 等效采样：示波器式可调（✅ 批次 3 已实现）

现状与原因：`host/panel_abz.py:259-264` 两个子图横轴写死 0–360°。AB 图把 256 个箱全铺在 360° ⇒ 每箱 1.4° ⇒ **太密**；UVW 图 6 个扇区铺满 360° ⇒ 每扇区 60° ⇒ **太稀疏**。

已确认做法 - 横轴**可缩放/平移**：AB 图可从 360° 缩到 90°/30°/10°/单箱邻域；UVW 图可缩到 1~2 个扇区；提供档位按钮 + 拖动平移 + "适应窗口"（双击恢复自动 = 整圈）。 - 横轴**口径可切换**：`电角度（°）` / `输入周期数` / `时间（ms）`（后者按当前 `f_in` 与 `inter` 换算）。 - 纵轴：三通道（A/B/Z、U/V/W）**通道高度与间距可调**（现在是写死的 2.2/1.1/0.0 台阶），并提供"叠加 / 分离"两种排布。 - UVW 图加**扇区边界线 + 当前状态编号标注**（`S1…S6`）；AB 图加"箱索引"次级横轴（缩放到单箱邻域时显示箱号）。 - 标注补全：横轴单位与当前范围、纵轴"逻辑电平 (0/1)"、图例、每通道当前值；游标读 `A→B 相位差`（缩放后仍给出全局值，避免误读）。

验收：`selftest_ui.py` 断言缩放档位切换后 `xlim` 正确、双击回到整圈、通道间距改变后曲线基线位置随之变化；离线断言 AB 缩到 30° 时可见箱数 ≈ 30/360×256。

R4 记录卡片「随动随止」（✅ 批次 1 已实现）

原因（已定位）：`host/panel_record.py:272` 的"校验错帧数"直接显示实时链路计数 `self._crc`，而主窗口每 0.5 s 喂一次（`host/panel_record.py:248-255` 的 `set_crc`），与记录是否在跑无关 ⇒ 停止后仍持续增长。

已确认做法 - 点击"停止记录"的瞬间，把卡片六项读数（时长、行数、文件大小、丢帧估计、校验错帧数、帧率）**全部冻结**在停止时刻的值，此后忽略任何刷新（链路计数照旧在后台更新，只是不再写进这张卡片）。 - 再次"开始记录"时**从 0 重新起算**（含校验错基线 `_crc_base`）。 - 状态灯：`● 记录中` / `已停止（已保存 N 行 → 路径）`。

验收：`selftest_ui.py` 新增用例——记录 → 停 → 人为增大 CRC 计数器并 tick 多次，断言六项读数一字不变；再开记录断言归零重算。

R5 通信日志降噪（✅ 批次 1 已实现）

原因（已定位）：主窗口每 `STATUS_PERIOD_S = 2.0` 秒轮询告警寄存器（`host/main_window.py:1027-1031`），每次回来无条件打印一行 `告警状态：…`（`host/main_window.py:849-855`）⇒ 每分钟 30 行，把有用信息冲走。

已确认做法（用户选择：保留 2 s 轮询，降为"调试"级别 + 加级别过滤开关） - 日志按级别分类：`故障 / 警告 / 信息 / 调试`；`告警状态：…` 这类周期性轮询结果**降为「调试」**，默认**不显示**。 - 日志卡片加**级别过滤**（勾选框或下拉：全部 / 只看警告及以上 / 只看故障），并加**清空日志**按钮；过滤只影响显示，内部仍保留最近 N 行。 - 状态**发生变化**（锁存位/`io_flags` 变化）时仍以「信息/警告」级别打印一条，保证不丢关键事件。 - 轮询周期保持 2 s（不改动链路行为），读失败连续多次合并为一条并计数，避免刷屏残留。

验收：`selftest_ui.py` 断言默认过滤下 2 s 轮询不产生可见行、状态变化产生一行、切换过滤后可见行数变化、清空按钮生效。

R6 每个小窗口可放大到全软件界面（✅ 批次 2 已实现：窗口内独占）

现状：所有卡片由同一工厂 `host/ui_kit.py:14 make_card(title, hint, hint_lines)` 生成，标题是普通 `QLabel`，无放大入口 ⇒ 一处改动即可覆盖所有卡片。

已确认做法 - `make_card()` 标题行改为 `QHBoxLayout`： `标题 + 弹簧 + [⛶ 放大]` 按钮；**双击标题**等效点击该按钮。 - 点击 `⛶` 进入**独占模式**：该卡片所在整行/整列之外的其它卡片 `hide()`，本卡片 `show()` 并撑满滚动区可视高度；按钮变 `⛶ 还原`， `Esc` 也可还原。 - 独占期间图表按新尺寸重排（字号、轴标签、刻度、通道间距按可用高度重算），退出时恢复原尺寸。 - 独占状态**不写入配置文件**（属于临时视图状态，重启回到常规布局）。

验收： `selftest_ui.py` 断言对每张卡片触发放大后"其它卡片不可见、本卡片高度 ≈ 可视高度"、再点还原后"所有卡片可见且几何与放大前一致"（保护 README「事实 7 固定几何」契约）。

交互默认值汇总（实现依据）

项	默认	说明
滚轮	永远滚页面（像素级平滑）	数字框/下拉框不吃滚轮；日志框/端口表白名单
图表缩放入口	档位控件 + 拖动平移 + **Ctrl+滚轮**缩放	裸滚轮永远滚页面；Ctrl+滚轮在图上缩放当前图表
波形横轴	自动跟随 `f_in`（1~2 周期，档位取整）	手动一调即锁定，双击恢复自动
波形纵轴	自动量程	手动档位 1.0/0.5/0.25/0.1 满量程
ABZ/UVW 横轴	自动 = 整圈（360°）	手动档位 180/90/30/10°，双击恢复整圈
通道排布	叠加（现样式）	可切"分离"，间距可调
记录停止	六项读数冻结	再开记录归零重算
日志级别	默认「信息」及以上	2 s 轮询降为「调试」，默认不可见
卡片放大	窗口内独占	`⛶` 或双击标题； `Esc` /再次点击还原
参数记忆	**要记忆**	示波器量程/通道间距/日志过滤/窗口尺寸 → `host/ui_prefs.json`，损坏时静默回落默认
导出	全貌导出（1950×1080 / 1950×990）**不变**	**另加**"按当前视图导出"按钮

分批实施计划（每批都跑离线 + 真机复验）

批次	内容	风险	验收
批次 1	R1（滚轮）+ R4（记录冻结）+ R5（日志降噪） ✅ **2026-10-13 已完成**	低（局部事件处理与显示逻辑）	`selftest_ui.py` 新增用例全绿 + 真机 S5/界面级真机核对 PASS
批次 2	R6（卡片放大独占） ✅ **2026-10-13 已完成**	中（触及布局几何契约）	上述 + README「事实 7」与界面自检几何契约同步更新并说明理由

批次	内容	风险	验收
批次 3	R2（波形示波器）+ R3（ABZ/UVW 示波器） ✅ **2026-10-13 已完成**（含 Ctrl+滚轮、`host/ui_prefs.json` 参数记忆、「按当前视图导出」）	较高（重画逻辑与坐标换算）	上述 + 真机 S3/S4 复核（含 A→B 相位差、箱覆盖率判据不回归）

批次 1 实施记录（2026-10-13）

需求	落地位置	验收证据
R1 滚轮一律滚页面	`host/main_window.py`：`_install_wheel_router()` / `eventFilter()` / `_wheel_is_ours()` / `_wheel_owner()` / `_wheel_zoom_target()` / `_route_wheel()` / `_scroll_page()`；`WHEEL_PX_PER_UNIT = 0.6`；日志框置 `wheelOwn` 白名单	界面自检 ⑬：端口下拉框上滚轮时页面 0 → 72 px 且选中项不变、数字框 `inter` 64 → 64 不变、日志框视口上滚轮滚动日志自己（页面 144 → 144）
R4 记录随动随止	`host/panel_record.py`：`_frozen` / `_freeze(st)` / `_show_frozen()`；`start_record()` 清冻结、`stop_record()` 立即冻结	界面自检 ⑪：停止后人为把 CRC 从 5 涨到 905、tick 三次，六项读数一字不变；状态灯「已停止」；重新开表读数归零重算（校验错=0、帧数=300）
R5 日志降噪	`host/main_window.py`：`LOG_RANK` / `LOG_COLOR` / `LOG_FILTERS` / `LOG_FILTER_DEFAULT`、`log_line(text, level)`、`_append_log()`、`_refresh_log_count()`、`_render_log()`、`_on_log_filter()`、`clear_log()`、`_status_signature()` + `_done_status()` 分级	界面自检 ⑫：状态没变时轮询行不可见（可见 1 → 1 条）；切「全部（含调试）」变 2 条；切回「信息及以上」回 1 条；「清空日志」生效
回归	——	`python host\selftest_ui.py` **42/42 PASS（exit 0）**（原 30 项 + 新增 12 项）；离线模块自检全绿：encoder 10、abz 15、export_png 11、recorder 13、signal 22、regs 2043 组 + 5 用例；`python -m compileall host` exit 0；真机截图核对 `python host\app.py --smoke host\_b1_ui.png --port COM3 --seconds 8 --height 1800` exit 0（1166×2456，日志卡片新增「显示级别」下拉 + 「清空日志」按钮 + 右侧「显示 N / 共 M 行」计数）

实现要点与踩坑（供批次 2/3 参考） - 滚轮事件在 Qt 里发给**滚动区视口**（`QPlainTextEdit.viewport()`），发给控件本体不会滚动它自己；界面自检因此第一次跑出 41 PASS / 1 FAIL（日志框滚动条 0），改发视口后 42/42。 - `_wheel_is_ours` 用 `self.isAncestorOf(obj)` 判断归属；下拉框弹出列表是独立顶层窗口，天然交给 Qt 处理。 - R5 的状态签名**故意不含幅值字段**（`amp_low_sq` / `amp_high_sq` / `stat_amp_sq` 每 2 s 都在小幅波动），否则等于没降噪。 - 批次 1 未触任何几何尺寸契约，`export_png.py` 的 1950×1080 / 1950×990 像素与 S5 判据不变。

批次 2 实施记录（2026-10-13）

需求	落地位置	验收证据
R6 卡片放大（窗口内独占）	`host/ui_kit.py`：`MAX_BTN_IDLE/MAX_BTN_ON/MAX_BTN_WIDTH`、`_TitleDblClick`（标题双击）、`_find_title()`（递归定位标题所在布局层）、`_has_stretch()`、`install_maximize(card)`（返回 `CardMax` 按钮）；`make_card()` 末尾自动安装；`host/theme.py` 新增 `QToolButton#CardMax` 样式；`host/main_window.py`：`_setup_card_maximize()`、`_card_canvases()`、`_card_title()`、`_toggle_card_max()`、`_enter_card_max()`、`_restore_card_max()`、`resizeEvent()`、`QShortcut(Esc)`	界面自检 ⑭ 共 10 条：11 张卡片标题行**全部**有「⛶ 放大」按钮；放大 ⑧ 后可见卡片 = 1（仅本卡）、卡片高 814 px / 可视 878 px、画布 250 → 657 px、按钮变「⛶ 还原」；按 Esc 后 11 张全回来、按钮回「⛶ 放大」、**卡片几何（可见性/最小高/最大高/实际高）与放大前逐值一致**、画布 657 → 250 px；双击 ⑨ 标题等效点击按钮
回归	——	`python host\selftest_ui.py` **52/52 PASS（exit 0）**（批次 1 的 42 项 + 本次 10 项）；`python -m compileall host` exit 0；`export_png.py` 的 1950×1080 / 1950×990 像素契约与 S5 判据未动

实现要点与踩坑（供批次 3 参考） - **不动布局结构**：独占只做三件事——藏其它卡片、把本卡最小高度抬到 `max(420, 可视高-120)`、把画布那 250 px 固定高临时解开（`setMaximumHeight(16777215)` + `setSizePolicy(Expanding, Expanding)`）；还原时按进入前记下的 `(isVisibleTo, minimumHeight, maximumHeight, sizePolicy)` 逐项恢复 ⇒ 天然满足 README「事实 7 固定几何」，无需重新 parent。 - 标题分布两种情形都要管：`make_card()` 造的卡片标题在竖向布局里（新建「标题+弹簧+按钮」水平行塞回原位）；各 panel 自己造的卡片标题常在一条水平标题行里（和「暂停刷新」之类开关并排）⇒ 直接在该行尾补弹簧+按钮。**首版只扫布局直接 item，wave 卡片拿不到按钮**，改成递归 `_find_title()` 后覆盖全部 11 张。 - `QShortcut` 在 **`PySide6.QtGui`**，不在 `QtWidgets`（写错会 `ImportError: cannot import name 'QShortcut' from 'PySide6.QtWidgets'`）；Esc 用 `setContext(Qt.ApplicationShortcut)`，窗口任意位置都生效。 - 事件过滤器实例必须挂在卡片上（`card._max_watcher`），否则被 GC 后双击标题失效。

批次 3 实施记录（2026-10-13）

需求	落地位置	验收证据
R2 波形示波器式可调	`host/panel_wave.py`：`X_SPANS_MS`（0.5…300 ms，共 10 档）、`Y_FULLS`（1.0/0.5/0.25/0.1 FS）、`AUTO_CYCLES=2.0`、`LAYOUTS/LAYOUT_TEXT`、控件 `cmb_tdiv` / `cmb_vdiv` / `cmb_layout` / `btn_auto` / `btn_export_view` / `lab_view`、视图协议 `nav_axes/x_span/set_x_span/x_center/set_x_center/y_pan/set_y_pan/reset_auto/scope_view`、`_auto_span_ms()`（按 `1000/f×2` 量化到档位）、`_y_lim()`、`_sync_controls()`、`_redraw` 重写（叠加/分离两种排布）	界面自检 ⑮：自动（20 kHz）满屏 0.500 ms；档位「满屏 20 ms」→ `xlim=-20…0`；「±0.25 FS」→ `ylim=-8192…8192`；分离排布 `ylim=±56885`；Ctrl+滚轮 20 → 16 ms；双击回自动

需求	落地位置	验收证据
R3 ABZ/UVW 示波器式可调	`host/panel_abz.py`：`X_SPANS_DEG`（10/30/90/180/360°）、`GAP_BASE=1.1`、`SPACING_RANGE=(0.6,2.5)`、控件 `cmb_xspan` / `cmb_layout2` / `sp_gap` / `btn_auto2` / `btn_export_view2` / `lab_view2`、视图协议同上 + `_chan_offsets()`（A/B/Z 基线随间距与叠加/分离变）、`_init_axes` 重写（按窗口设 xlim、y 刻度标签 A/B/Z 与 U/V/W、UVW 60° 分隔线只在整圈时画）、`_redraw` 无数据也画窗口	界面自检 ⑮：自动 360°；「满屏 90°」→ `xlim=135…225°`；`sp_gap=2.0` → A/B/Z 基线 4.40/2.20/0.00、纵轴 -0.30…5.60；分离 → -0.30…10.00；Ctrl+滚轮 90 → 72°；双击回 360°
Ctrl+滚轮 / 拖动平移 / 双击复位	`host/ui_kit.py` 新增 `class ScopeNav(QObject)`（`canvas.installEventFilter`、`canvas.setProperty("wheelZoom", True)`、`ZOOM_STEP=0.8`、`_px_ratio()` / `_frac()` 高 DPI 换算、以光标为中心缩放、整体 try/except 不把异常冒进事件循环）；`host/main_window.py` 的全局滚轮过滤器用 `_wheel_zoom_target()` 把 Ctrl+滚轮让给图表	界面自检 ⑮：Ctrl+滚轮后**页面滚动条不动**、图量程按档位变；双击 `canvas` 回自动；编辑不再有"Ctrl+滚轮改错东西"
参数记忆 `host/ui_prefs.json`	新模块 `host/ui_prefs.py`（13 项自检 PASS）：`DEFAULTS`（window/log/wave/abz 四段）、`load/save/data/section/raw/number/choice/put/window_size/_merge`，键值夹取与错类型回落；`host/main_window.py` 开窗 `up.window_size()` + 日志级别、关窗 `_save_prefs()`（先 `_restore_card_max()` 再存几何）	界面自检 ⑯ 5 条：改量程/间距/日志级别/窗口后重开窗口逐值恢复（`wave 50 ms / abz 90° / 间距 1.6 / 日志 debug / 窗口 1000×700`）；把参数文件写成坏 JSON ⇒ 回落默认且不崩；自检把 `up._path_override` 指到临时目录，**不碰用户的 `host\ui_prefs.json`**
「按当前视图导出」	`host/export_png.py` `export*(..., view=)`（wave 用 `view["xlim"]/["ylim"]`、分离排布自动上下平移；abz 用 `view["offsets"]/xlim/ylim`，60° 分隔线只在 ≥359° 时画；meta 加「量程」行 + 文件名/标题带「（当前视图）」）；`host/panel_record.py` `finish_export()` 认 `*_view`；`host/main_window.py` 接两个 `export_view_requested` 信号	界面自检 ⑰ 5 条 + `export_png.py` 自检 **[12]**：`encoder_wave当前视图_*.png` 1950×1080 / `encoder_abz_uvw当前视图_*.png` 1950×990（用非白像素数证明视图真生效：1307277 → 1311492、ABZ 763489 → 763810）；全貌导出仍不带「当前视图」；空数据 + 视图不崩

需求	落地位置	验收证据
回归	——	`python host\selftest_ui.py` **73/73 PASS（exit 0）**（批次 1 的 42 + 批次 2 的 10 + 本次 21）；`python -m compileall host` exit 0；离线自检全绿：encoder 10、abz 15、export_png **12**、recorder 13、signal 22、regs 2043 组 + 5 用例、ui_prefs 13；真机 `selftest_s3.py --port COM3` PASS、`selftest_link.py --port COM3 --seconds 5` PASS、`selftest_ui.py --real-port COM3 --seconds 6` PASS（界面取 60 954 帧 = 记录器写 60 954 行，差 0）、`selftest_s4.py --port COM3` PASS（11 项；1 项因当前信号源工况不可测而跳过，见下）

实现要点与踩坑 - **时基档位必须细到亚毫秒**：原来最小 5 ms，20 kHz 输入会显示 100 个周期（用户抱怨"方波太密"）⇒ `X_SPANS_MS` 增到从 0.5 ms 起（1 帧 = 0.1 ms，最细 5 帧）。 - 自动量程按 `1000/f_in × 2` 周期算完再**量化到档位**：否则轴标签每帧都在跳，读不稳。 - Ctrl+滚轮要在两处配合：`ui_kit.ScopeNav` 吃事件，`main_window` 的全局滚轮过滤器靠 `canvas` 上的 `wheelZoom` 属性**让路**（不然图表缩放与页面滚动会打架）。 - `export_png.py` 里 `ylim` 原来只在 `if n:` 分支赋值 ⇒ 空数据 + 视图导出抛 `cannot access local variable 'ylim'`（被自检 [5] 抓到），已提到分支外。 - **真机 S4 的 A→B 相位差在 100 kHz 信号源下不可测**：实测输入 99 999.5 Hz、inter=32 ⇒ AB 输出 3.2 MHz（周期 312 ns ≪ 帧周期 100 µs），帧里的 AB 电平与角度遥测对不上——256/256 箱的 A/B 平均值全是 0.50（正常方波几乎全为 0/1）。`host/abz.py` 因此新增"电平与角度不相关"判据（`ab_muddy`，进 notes 并把 `phase_ab_deg` 置 None，不再报假 0.0°），`host/selftest_s4.py` 把该项标为**跳过**并在结果里明示。同一份代码在 19 999.901 Hz 下实测 −90.0°、在 1 kHz 下也过关，`abz.py` 的离线 15 项自检仍断言 ±90° ⇒ 不是相位算法回归。**要看相位请把信号源调到 20 kHz 以内。** - 顺带修掉一个误导性报错：`host/link.py` 的 `_open_error_text()` 原来先匹配 `"could not open port"`，把"端口被占用（`PermissionError(13, 拒绝访问)`）"报成"找不到这个串口（板子被拔了？）"——真踩过，害我查了半天硬件。现在按内层 errno 分派，并在 `SerialWorker.run()` 里对打开失败重试 3 次（间隔 0.6 s），覆盖"上一个脚本刚退出、句柄还没释放"的常见情形。

全部批次完成后的收尾：三个批次均已完成，收尾证据见 `host/README.md`、`docs/DEV_STATUS.md` §1.70/§1.72/§1.73 与本节各表；界面自检 73 项、离线模块自检 101 项与真机 S3/S4/S5 + 界面级真机核对全绿。

已确认（2026-10-13 用户拍板）

1. **要 Ctrl+滚轮缩放**：裸滚轮只滚页面，Ctrl+滚轮缩放当前图表；面板档位控件同样保留。
2. **参数要记忆**：示波器量程、通道间距、日志过滤级别、窗口尺寸存 `host/ui_prefs.json`；文件缺失或损坏时静默回落默认值。
3. **导出另加"按当前视图导出"按钮**：原有全貌导出（1950×1080 / 1950×990）保持不变，S5 像素判据不动。
4. **实施顺序：批次 1 → 2 → 3**，每批完成后跑离线自检 + 真机复验，并更新本文件勾选与 `README` / `DEV_STATUS`。

实现过程中的小决定（不等确认，随批次说明）：游标测量作为可选项默认关闭；`ui_prefs.json` 加入 `.gitignore`；手动锁定后在图角显示"手动"小字标识。

批次 4：放大后的界面（排版 / 滚轮 / 设置挡位）— 需求整理稿（待用户确认）

- **提出**：用户于 `m08815` 提三条，明确要求**先沟通需求、确认后再实施**
- **整理日期**：2026-10-14
- **范围**：`host/ui_kit.py`（`ScopeNav` / `install_maximize`）、`host/main_window.py`（`_enter_card_max()` / `_restore_card_max()` / 滚轮路由 `_route_wheel()`）、`host/panel_wave.py`、`host/panel_abz.py`、`host/ui_prefs.py`、`host/theme.py`
- **不动**：协议/解析层（`link.py` / `encoder.py` / `abz.py` / `alarm.py` / `regs.py`）；`export_png.py` 的 1950×1080 / 1950×990 像素契约
- **硬约束**：不新增第三方依赖；界面全中文；`selftest_ui.py`（现 73 项）与离线自检（现 101 项）保持全绿；**R1 的"普通页滚轮滚页"与 R6 的"放大还原后几何逐值复原"两条既有契约不得回归**

用户原话（`m08815`）

> 清理整理完之后，对软件进行UI优化，放大后的界面UI需要进行优化，调整布局排版，再放大后的界面，就不需要"在软件全界面处都可以通过滚动鼠标滚轮滚轮进行界面顺畅上下滑动"这项功能了。放大后的界面，一些设置挡位等可以更加方面和细致一些，优化优化设置逻辑，变得更加方便一些，以便适应更加多元化的显示需求。

现状取证（2026-10-14 • 离线 offscreen 1600×900）

取证脚本（临时，跑完即删）：`host/_shot_max.py`（抓图 + 量几何 + 造滚轮事件）、`host/_probe_layout.py`（量控件在**卡片内**的真实位置）。实测：

项	整页	放大 ⑧ 波形	放大 ⑨ ABZ
滚动区 内容高 / 视口高	2597 / 878（可滚 1719 px）	**878 / 878 ⇒ 滚不动**	**878 / 878 ⇒ 滚不动**
放大卡片	——	1554×807（独占整窗）	1554×807（独占整窗）
子图宽度	时域 ≈630 / 李萨如 ≈430，**中缝 ≈240 px**	同上（图高 557）	ABZ ≈715 / UVW ≈735（较合理）
读数格	2×3 网格**拉伸铺满** 1554 px：左列 x≈30…140、右列 x≈798…910 ⇒ 中缝 ≈660 px	同上	底部统计表同理：4 组分散在 x≈30 / 370 / 790 / 1130
控制行	控件全挤在左侧（卡片内 x=53…596），`导出当前视图` 单独在 x=1431	同上	同上

结论：放大功能**本身是好的**（卡片确实独占、几何可逐值复原、无残留滚动条，R6 契约成立）。用户不满意的是**放大后没有把空间重新组织**：

1. 读数格与统计表被"拉宽铺满"，中间留大片空白，标签与数值离得远，反而不如窄屏好读；
2. ⑧ 的两张子图之间有 ≈240 px 中缝（≈15% 卡片宽度是死的）；
3. 控件仍挤在左上角一行，而放大后明明有整屏空间可以给更大、更好点的目标；
4. **放大态页面已无可滚动内容**（滚动条范围 0），此时 `host/main_window.py:720-730` 的 `_route_wheel()` 仍然无条件 `return True` **吃掉滚轮** ⇒ 放大态滚轮等于失效（用户要求的"放大后不需要全界面滚轮滚页"正

是指这一点）。实测造滚轮事件：整页时裸滚轮发到波形画布 **页面 0 → 72 px**（R1 生效）、发到滚动区视口 72 → 144 px，`Ctrl+滚轮` 时不滚页（144 → 144，交给 `ScopeNav`）；放大 ⑧ 后三种发法都是 **0 → 0**（范围 0，滚不动）；

5. 挡位是固定粗档的 `QComboBox`，没有步进按钮、没有快捷键、没有连续微调与"自定义"显示，非档位值无处输入。

候选做法（请拍板：每项选一个或组合）

R7 放大后的排版

选项	做法	代价
A	**右侧控制列**：放大时把控制条 + 读数竖排进右侧固定宽（建议 280 px）一列，图表占满左侧剩余宽高（示波器式）	需在放大/还原间切换两套布局，还原必须逐值复原（R6 契约）
B	**上控制条 + 定宽读数块 + 子图按比例分配**：读数块定宽不拉伸（靠左/居中）；⑧ 两子图按 2:1 分配消掉中缝；⑨ 统计表按定宽列分组	改动集中在 panel 的布局代码，风险最低
C	**沉浸式**：图表满幅，控制/读数收成一条可自动隐藏的浮动工具条（鼠标移入才展开）	交互最复杂，最易挡图

> 建议：**B 作基础（普通页也跟着受益）+ A 作放大专属增强**（放大切右侧列，还原回上方行）。

R8 放大后的滚轮

选项	含义
A	**关闭滚轮滚页**（用户原话）：放大态不再吃滚轮、页面不滚；图上滚轮也不做事
B	**滚轮 = 缩放图表横轴**（免按 Ctrl）；`Ctrl+滚轮` = 更细的微调（如 0.98×）或纵轴缩放 ⇒ 符合示波器习惯
C	**滚轮 = 切挡位**（一格一档，如时基 10→20→50 ms），适合逐档比对

> 建议：**B**（同时满足 A 的语义：不再滚页面）。**普通页 R1 行为保持不变。**

R9 放大后的设置（挡位）细化与便捷化

1. **挡位序列加密**：时基 `0.5/1/2/3/5/7.5/10/15/20/30/50/75/100/150/200/300 ms`；幅值 `1.0/0.75/0.5/0.25/0.1/0.05 FS`；ABZ/UVW 角度 `5/10/20/30/45/60/90/120/180/270/360°`
2. **步进按钮 + 键盘**：◀/▶（放大态大目标）逐挡步进；`←/→` 时基、`↑/↓` 幅值/角度、`Shift+←/→` 更细一档、`R` 恢复自动、`Space` 暂停、`E` 导出当前视图、`Esc` 还原、`F` 全屏
3. **无级微调**：`Ctrl+滚轮` 连续缩放时挡位框显示"自定义 ×××"（不再硬贴档位）；`Shift+滚轮` 0.98× 细步
4. **数字直填**：时基/幅值/角度旁给小输入框，可直接输 `33`、`7.5` 这类非档位值（内部夹取到合理区间）
5. **显示方案预设**：一键切「概览 / 单周期 / 精细 / 整圈」，可命名保存进 `ui_prefs.json`，放大态工具条上给按钮
6. **记忆**：每卡上次挡位/排布继续记忆；**放大状态本身不写盘**（沿用 R6 约定）

> 建议顺序：**1 + 2 先做**（收益最大、风险最低）→ 3 + 4 → 5 视需求。

已确认（2026-10-14 用户拍板）

编号	问题	用户选择
Q1	R7 排版	**B 打底 + A 放大专属**（普通页先把读数块定宽、⑧ 子图按比例分宽、⑨ 统计表分组；放大态再把控制+读数竖排到右侧一列）
Q2	R8 放大态滚轮	**滚轮 = 缩放图表横轴，Ctrl+滚轮 = 微调**（同时满足"放大态不再滚页面"）
Q3	R9 挡位细化	**只做「挡位序列加密 + ◀▶ 步进按钮 + 键盘快捷键」**；无级微调/自定义显示、数字直填、显示方案预设**本轮不做**（留待后续按需）
Q4	全屏与切卡片	**都要**：真·全屏（`F` / `F11`）+ 放大态内直接切卡片
Q5	放大态细节	**收起那几行提示文字 + 读数加大字号**

键位冲突的处理（`←/→` 归时基）： Q3 已把 `←/→` 定为时基步进（示波器上最高频的操作），而 Q4 的"切卡片"若也用 `←/→` 会打架 ⇒ 定为：`←/→` = 时基步进、`↑/↓` = 幅值/角度档、`Shift+←/→` = 更细一档、`PgUp/PgDn` 与 `Ctrl+←/→` = **放大态内上一张/下一张卡片**，并在放大态状态栏写明。

实施方式（确认后照此执行）

批次 4 拆两小批：**4a** = R8 + R7-B + R9①（挡位加密）与 R9②（步进按钮 + 快捷键）+ R11（收起提示、读数加大）+ R10（全屏、放大态切卡片）；**4b** = R7-A 右侧控制列。每批跑 `selftest_ui.py` + 离线自检，更新 `host/README.md`、`docs/DEV_STATUS.md` 与本文件勾选；串口在位时补真机 S3/S4 复验。

R7 放大后的排版（✅ 批次 4a 打底 + 4b 增强，两小批全部落地）

- **B 已落地（4a，普通页也跟着受益）**：读数块定宽不拉伸（`host/panel_wave.py:162`、`host/panel_abz.py:174` 的 `grid.setColumnStretch(2, 1)`）、⑧ 两子图按 2:1 分宽（`host/panel_wave.py:175-177` 的 `fig.add_gridspec(1, 2, width_ratios=(2.0, 1.0))`）、⑨ 六状态表两组合并靠左（`host/panel_abz.py:206-207` 的 `tbl.setColumnStretch(6, 1)` + `setColumnMinimumWidth(3, 36)`）。
- **A 已落地（4b）**：放大态把「控制条 + 量程行 + 读数」整块竖排进**右侧固定 300 px 一列**（`SIDE_MAX_W = 300`），图表（⑧ 两子图 / ⑨ 双图）独占左侧 1216 px 并吃到整高；读数在放大态改**一行一条的竖排**（右列放不下两列）。实现方式是面板顶层从 `QVBoxLayout` 改成 `QGridLayout`，由新增的鸭子型方法 `set_max_layout(on)` 在两态间搬格子（`host/panel_wave.py`、`host/panel_abz.py`），由 `_enter_card_max()` / `_restore_card_max()` 调用；还原仍走逐值复原契约。

验收证据（离线 offscreen 1600×900，放大卡片同为 1554×814）

项	批次 4a 之前	批次 4a 之后
⑧ 读数格	左列 x≈30…140、右列 x≈798…910（中缝 ≈660 px，被拉宽铺满）	全部落在 x≈14…580（`v_rmean` x=96/w=242、`v_board` x=188、`v_span` x=430/w=150）
⑧ 子图	时域 ≈630 / 李萨如 ≈430，**中缝 ≈240 px**	时域 834 px / 李萨如 417 px（2:1），**中缝 138 px**
⑨ 六状态表	4 组分散在 x≈30 / 370 / 790 / 1130	两组紧挨：FieldName 列起点 `[14, 254, 280, 320, 378]`

项	批次 4a 之前	批次 4a 之后
放大态控制/读数（4b）	挤在图上方一行，右半屏空着	右列 300×759 px 竖排；图表左移占 `1216×759`（⑧）/ `1216×675`（⑨）
放大态读数（4b）	2×3 两列，长字段名被挤成"板上 r / 上位"	一行一条竖排，`板上 r / 上位机 r`、`AB 边沿率（理论）` 都能完整显示

R8 放大后的滚轮（✅ 批次 4a：滚轮缩放横轴 + Ctrl 微调）

- `host/main_window.py` 的 `_route_wheel()`：**放大态**（`_max_card is not None`）目标是图表（带 `wheelZoom` 属性）⇒ `return False` 让 `ScopeNav` 处理；放大态滚轮落在别的控件上 ⇒ `return True` **吃掉**（既不滚页、也不偷偷改数字框/下拉框）；**普通页契约一字未改**（裸滚轮滚页面、只有 `Ctrl+滚轮` 归图表）。
- `host/ui_kit.py` 的 `ScopeNav` 新增 `_plain_zoom` 与 `set_plain_wheel_zoom(on)`：放大态裸滚轮 = `ZOOM_STEP=0.8` / 120 单位缩放横轴，`Ctrl+滚轮` = `FINE_STEP=0.98` 微调。`_enter_card_max()` 打开、`_restore_card_max()` 关闭。

验收证据（自检 ⑱）：放大态裸滚轮 `10.00 → 8.00 ms`；`Ctrl+滚轮` `×0.9800`；滚轮落在滚动区视口 ⇒ 量程 7.84 ms 不变、页面仍 0 px；还原后裸滚轮 `页 0 → 72 px` 且量程不动。

R9 放大后的设置挡位细化与便捷化（✅ 批次 4a 做 ① ②；③④⑤ 本轮不做）

- **① 挡位加密**：`host/panel_wave.py` `X_SPANS_MS` 10 → **17 档**（0.5/1/1.5/2/3/5/7.5/10/15/20/30/50/75/100/150/200/300 ms）、`Y_FULLS` 4 → **7 档**（1.0/0.75/0.5/0.35/0.25/0.15/0.1 FS）；`host/panel_abz.py` `X_SPANS_DEG` 5 → **11 档**（5/10/20/30/45/60/90/120/180/270/360°）。
- **② 步进按钮**：`host/ui_kit.py` 新增 `STEP_BTN_WIDTH=26` + `step_button(text, tip)`（`objectName="StepBtn"`、`setFocusPolicy(Qt.NoFocus)` 免得抢方向键），样式在 `host/theme.py` 的 `QToolButton#StepBtn`（15 px 加粗、24×24 起）。波形 4 个（◀/▶ 时基夹住 `cmb_tdiv`、`−`/`+` 幅值夹住 `cmb_vdiv`）、ABZ 2 个（◀/▶ 夹住 `cmb_xspan`）；到端点自动置灰（`panel_wave._sync_step_btns()`、`panel_abz._sync_controls()` 里 `setEnabled(i > 0)`）。步进由 `step_x()/step_y()` 实现：整档走档位表、`fine=True` 走 `FINE_STEP=0.9` 连续微调（ABZ 间距微调 `FINE_GAP=0.05`）。
- **② 键盘快捷键**：`host/main_window.py` `_setup_keys()` 建 17 条 `QShortcut`（`Qt.WindowShortcut`）——`←/→` 时基、`↑/↓` 幅值（⑨ 为通道间距）、`Shift+方向键` 微调、`R` 恢复自动、`Space` 暂停刷新、`E` 导出当前视图、`F`/`F11` 全屏、`PgUp/PgDn` 与 `Ctrl+←/→` 放大态切卡片。焦点落在 `QLineEdit`/`QAbstractSpinBox`/`QPlainTextEdit`/`QTextEdit` 时**全部让键**（`_on_focus_changed` 挂 `QApplication.focusChanged`）；方向键本身无声，调完把新量程写进状态栏 1.5 s（`_kbd_note`）。
- **本轮不做**：③ 无级微调 + "自定义"显示、④ 数字直填、⑤ 显示方案预设（用户 Q3 明确留待后续）。

验收证据（自检 ⑱）：档数 17/7/11；步进按钮 6 个且样式类 `StepBtn`；`10 ms → 7.50 ms`；微调 `9.000 ms`；端点 `◀=False`；`1.0 → 0.75 FS`；方向键 `7.50 ms` 且状态栏「时基：横轴 7.5 ms（手动）· 纵轴 手动」；`Shift+←` `6.750 ms`；`R` ⇒ `x=auto y=auto`；焦点在 `sp_inter` 时按 `←` 量程保持 10.00 ms。

R10 真•全屏与放大态切卡片（✅ 批次 4a）

- `_toggle_fullscreen()`：`F` / `F11` 进全屏前记住 `isMaximized()`，退出时回到最大化或普通窗口（`Esc` 仍只负责还原卡片放大）。
- `_cycle_card_max(delta)`：顺序 = `self._cards`，先 `_restore_card_max()` 再 `_enter_card_max(下一张)` ⇒ 每张卡片的几何契约照旧逐值复原；未放大时按键只给状态栏提示。

验收证据（自检 ⑱）：`F` 进/出全屏 `进=True 出=True`；`PgDn` ⇒ 放大卡片变为「⑨ ABZ / UVW 等效采样」。

R11 放大态收起提示文字与读数放大（✅ 批次 4a）

- 进入放大：卡片内 `property("maxHide")` 的 `QLabel`（两个面板的 `lab_hint`）隐藏；读数 `objectName=="BigValue"` → `font-size:34px`、`"BigValueSmall"` → `27px`、`"FieldName"` → `14px`；改过的东西连同旧 `styleSheet()`/可见性存进 `self._max_ui_saved`，`_restore_card_max()` 逐值退回 ⇒ R6「还原后逐值复原」的契约从几何扩展到样式。

验收证据（自检 ⑱）：放大态 `hint 可见=False`、读数 `（空）→ font-size:27px;`；还原后 `hint 可见=True`、读数样式回到空。

批次 4a 实施记录（2026-10-14）

需求	落地位置	验收证据
R7-B 排版	`host/panel_wave.py`（`:162` 读数格、`:175-177` gridspec 2:1）、`host/panel_abz.py`（`:174` 读数格、`:206-207` 六状态表）	自检 ⑱ 三条；离线实测见 R7 表（读数中缝 660 → 全部 ≤580 px、子图中缝 240 → 138 px、统计表 4 组 → 2 组紧挨）
R8 放大态滚轮	`host/main_window.py` `_route_wheel()` + `host/ui_kit.py` `ScopeNav._plain_zoom/set_plain_wheel_zoom/FINE_STEP`	自检 ⑱ 四条（裸滚轮 10→8 ms、Ctrl 微调 ×0.9800、非图表处量程与页面都不动、还原后页 0→72 px）
R9① 挡位加密	`host/panel_wave.py` `X_SPANS_MS` / `Y_FULLS`、`host/panel_abz.py` `X_SPANS_DEG`	自检 ⑱ 两条（17 档 / 11 档）
R9② 步进按钮	`host/ui_kit.py` `step_button()`、`host/theme.py` `QToolButton#StepBtn`、两个面板的 `step_x()/step_y()`	自检 ⑱ 五条（6 个按钮、10→7.5 ms、微调 9.000 ms、端点置灰、1.0→0.75 FS）
R9② 键盘	`host/main_window.py` `_setup_keys()/_on_focus_changed()/_kbd_*()`	自检 ⑱ 五条（← 7.50 ms + 状态栏、Shift+← 6.750 ms、R 回自动、数字框聚焦时让键）
R10 全屏/切卡片	`host/main_window.py` `_toggle_fullscreen()/_cycle_card_max()`	自检 ⑱ 两条

需求	落地位置	验收证据
R11 收起提示/读数放大	`host/main_window.py` `_enter_card_max()/_restore_card_max()` + `host/panel_wave.py`、`host/panel_abz.py` 的 `lab_hint.setProperty("maxHide", True)`	自检 ⑱ 三条（含还原逐值复原）
回归	——	`python host\selftest_ui.py` **95/95 PASS（exit 0）**（原 73 + 新增 ⑱ 22 项）；`python -m compileall -q host` exit 0；离线自检全绿：encoder PASS、abz 15、export_png PASS、recorder PASS、alarm 22、regs PASS、ui_prefs 14；`python host\app.py --smoke host\_b4_smoke.png --seconds 3` ⇒ 截图成功（1586×2597，88100 B）

实现要点与踩坑（批次 4a）

- **放大态几何取证必须"让事件循环跑一轮再量"**：在 `_toggle_card_max()` 同一次调用里立刻 `repaint()+grab()+量几何`，量到的是重排前的中间态（ABZ 卡片 y=1052、滚动条范围 0，截图看着像"放大后错位"）——不是 bug。等 600 ms 后一切正常（内容高 878 = 视口高 878 ⇒ 滚动范围 0）。
- **放大量控件位置别用 `mapTo()`**：`mapTo(win, w.geometry().topLeft())` 对这种嵌套卡片会算出漂移值（`btn_export_view` 报 x=2878，而卡片只有 1554 宽）；以**卡片内** `geometry()` 为准（实测该按钮在卡片内 x=1431，正常）。
- **读数 `objectName` 有两档**：`BigValue`（34 px）与 `BigValueSmall`（27 px），只判前者会漏；⑨ 六状态表是 `QGridLayout` **不是** `QTableWidget` ⇒ 自检只能 `findChildren(QGridLayout)` 后看 `columnStretch(6)`。
- **方向键必须"让键"而不是"吞键"**：`QShortcut` 用 `Qt.WindowShortcut`，再挂 `QApplication.focusChanged` 在输入控件聚焦时把 17 条全 `setEnabled(False)`；否则用户在数字框里按 `←` 会同时改量程和光标位置。
- **`Ctrl+←/→` 与 `PgUp/PgDn` 双绑切卡片**：`←/→` 已被时基占用，给习惯用方向键的人留 `Ctrl` 组合。

批次 4b 实施记录（2026-10-14）

需求	落地位置	验收证据
R7-A 放大态右列	`host/panel_wave.py`、`host/panel_abz.py`：顶层由 `QVBoxLayout` 改 `QGridLayout`；新增 `SIDE_MAX_W=300`、`self.side`（`QWidget` + `QVBoxLayout`，装 `_ctrl_grid` → `lab_view` → 读数格）、`_fill_ctrl(vertical)`、`_reflow_readouts(vertical)`、`set_max_layout(on)`；`host/main_window.py` `_enter_card_max()/_restore_card_max()` 各加一行鸭子型调用	自检 ⑲ 十条；离线实测：放大卡片 1554×811、⑧ 图表 `1216×759 @x14` + 右列 `300×759 @x1240`、⑨ 图表 `1216×675` + 右列 `300×675`；还原后右列宽 897/1526、图表回其下方且高度回 250/240
放大态读数竖排	`host/ui_kit.py` `field()` 存 `v._row_lay`；两个面板 `_rd_grid` / `_rd_vals` / `_reflow_readouts()`	自检 ⑲ 第 10 条（`v_span.x == v_rmean.x` 且 `v_span.y > v_rmean.y`）；截图中 `板上 r / 上位机 r`、`AB 边沿率（理论）` 不再被截断
步进按钮可读性	`host/theme.py` `QToolButton#StepBtn`（15 px 加粗、24×24 起、`:disabled` 用浅灰）、`host/ui_kit.py` `STEP_BTN_WIDTH=26`	截图中 ◀ ▶ 与 `– +` 明显；端点置灰仍生效（幅值=自动时 `–` 灰、角度窗口=整圈 360° 时 ▶ 灰）

需求	落地位置	验收证据
回归	——	`python host\selftest_ui.py` **105/105 PASS（exit 0）**（原 73 + ⑱ 22 + ⑲ 10）；`python -m compileall -q host` exit 0；`encoder` / `abz` / `export_png` / `alarm` / `regs` / `ui_prefs` 全 exit 0

实现要点与踩坑（批次 4b）

- **★ 重排读数必须"按值反查"再摘行**：第一版 `_reflow_readouts(False)` 按目标格子 `itemAtPosition(r, c)` 删，但放大态的排列与普通态不同 —— 目标格子里是**别的**读数行，于是摘错行 + 留下残行，普通态布局比基线高 32 px，R6 的「还原后卡片几何与放大前逐值一致」直接 FAIL（103/104）。改成 `idx = g.indexOf(v._row_lay)` 反查后逐值复原，`图 y=195` 与基线一致。
- **`QGridLayout` 里搬控件比 `QVBoxLayout` 稳**：`layout.removeWidget()` + `addWidget(w, r, c)` 可把同一个 `side` 容器在两态间搬位置；别忘了同时改 `setRowStretch()`（放大态让第 2 行吃满、普通态让第 3 行吃满），否则图表长不高。
- **右列 300 px 容不下两列读数**：`uk.field()` 的名字列 76 px + 数值 `setMinimumWidth(150)` ⇒ 两列要 ≈464 px；放大态必须竖排（一行一条），否则出现"板上 r / 上位"这种截断。
- **两态切换要幂等**：`_fill_ctrl()` / `_reflow_readouts()` 每次都先清空（`takeAt(0)` / `indexOf` 反查）再重排，避免"进放大→还原→再进放大"累积多出来的布局项。
- **offscreen 截图要看中文得给字体目录**：`QT_QPA_FONTDIR=C:\Windows\Fonts`（否则渲染成方框；`QT_QPA_PLATFORM=offscreen` 时默认找不到 `E:/Anaconda/Library/lib/fonts`）。
- **`recorder` 的写入性能自检已改成"同机自校准"**：原来是固定门槛 `rate > 150000`（`host/recorder.py:678`，历史记录 20.3–21.1 万行/秒），但这条路径是纯 CPU 的逐行字符串格式化，本机实测只有 **3.7–5.6 万行/秒**（进程 CPU 占用 100%、无竞争；这是机器慢，不是回归 —— 同一台机器在空载时同项实测 19.1 万行/秒、上限 32.2 万行/秒，比值 0.59）。现改为：先量"同机纯格式化上限"（5.4–9.3 万行/秒），要求记录器 **≥ 它的 35%**（比值实测 0.60–0.81），并保留"实机 10 kHz 只要 1 万行/秒"的 **2 倍硬底线 20,000 行/秒** —— 机器快慢不再影响判定，而"每行都 fsync""多拼一个 512 字符字段"这类真回归仍会被底线或比值抓住（已知局限：本机格式化成本就有 12 µs/行，因此 2 倍以内的格式化退化这项测不出来，写在这里备案）。

R12 实机复查修正（✅ 2026-10-14，用户接上板子后）

背景：用户把板子接到 **COM3**（CH340 @2 Mbaud，固件 v12、`inter=256 div=1`、只开 ABZ）后要求"全面复查软件运行效果、 全面检查 bug 和问题"（`m09768`）。S1–S5 与 10 分钟长稳全部 PASS，同时**修掉 3 个真问题**（详细取证与 RTL 依据见 `docs/DEV_STATUS.md` §1.81）。这三条都属于"界面/自检给出的信息不准"，正好落在本文件的范围内：

需求	问题	落地位置	验收证据
R12-1 UVW 关着时不再误报 FAIL	`selftest_s4.py` 把"无非法 UVW 状态""六个合法状态都出现"无条件执行，而板上 `0x0109 bit7=0`（U/V/W 恒 0）⇒ 每次跑都报 2 项 FAIL	`host/selftest_s4.py`（两项一并 gate 在 `out_bits.get("uwven")` 上，否则进 SKIP 并打印原因）	真机 `selftest_s4.py --port COM3` ⇒ **PASS（8 项 + 2 项按需跳过）**

需求	问题	落地位置	验收证据
R12-2 锁存位 `0x0405` b5（幅值越限）要有中文告警行	卡片⑦只在 `0x053A` 的实时位与遥测上判断，`STAT_LATCH=0x24`（含 b5）时显示"幅值越限 正常"，只有底部 `0x0405=0x24` 能看出异常	`host/alarm.py::evaluate()`（实时位优先 ⇒ `fault`；仅历史锁存 ⇒ `warn`「已锁存 b5」），`--selftest` 补 3 条	`alarm.py --selftest` **25/25**（原 22）；真机 `--force-fault` 造 `0x08` 再清成 `0x00` 全程 PASS
R12-3 "采样不足"要说清真正原因，并在图上标出来	文案把成因写成"输入频率太高…降到 20 kHz 以内"（真机 `f_in≈1.0 kHz` 也 muddy，真因是 `bins(256) ≤ 4×inter(1024)`）；更糟的是面板上 A/B 仍画出一条混叠出来的"假方波"且无任何标记	`host/abz.py`（分因诊断 + 可行做法：`inter ≤ 箱数/8`，或箱数够时才建议提箱数）、`host/panel_abz.py::_redraw()`（muddy 时子图标题改「采样不足：A/B 波形不可信，只看 Z 与读数」+ A/B `alpha=0.30`）、`host/selftest_s4.py`（跳过说明改为引用分析器原话）、`host/selftest_ui.py`（⑨ 节补 2 项）	`abz.py --selftest` 18/18；`selftest_ui.py` **107/107**（原 105）；截图确认图上出现"不可信"标题且 A/B 变淡
R12-4 建议阈值与自检覆盖（自查补）	① 第一版建议写成"`inter` 降到 ≤ 箱数/4"，但判据是 `bins ≥ 8×inter` ⇒ 按该值调到 `inter=64` / `bins=256` 仍 muddy（无效建议）；② 离线造帧器 `synth_frames()` 用同一个 16 位角度算 A/B 与角度遥测 ⇒ `ab_muddy` 永远为假，"采样不足"路径从没被自检覆盖	`host/abz.py`：建议改"≤ 箱数/8（保证每箱跨 ≤0.5 个 A/B 周期）"；`synth_frames(..., fine_bits=)` 新增参数（A/B 用细分相位、`angle` 仍 16 位，等价真实板子；`fine_bits=0` 保持旧行为）	`abz.py --selftest` **15 → 18 项**（新增：muddy 判为真、文案按"箱数不够"分因且含"箱数/8"、不再一口咬定"输入频率太高"）

顺带确认不是问题的两处（写在这里免得下次重复查）：卡片③/⑩的"校验错 0.39%"是**串口链路逐帧 XOR**（与 BiSS 帧 CRC 无关），"丢帧估计 0.77%" = 校验错帧 + 重同步吞掉的帧占应到帧时隙的比例；`Ctrl+滚轮` 缩放后组合框按"最近档位"显示（显示"满屏 50 ms"、 实际 49.152 ms）是本轮用户明确不做的"无级微调/自定义显示"的已知表现，功能上不影响。

R13 一键使能 ABZ / UVW + 三个"为什么"的成因说明（✅ 2026-10-14 晚）

用户原话（`m10906`）：「为什么ADC会过载锁存？为什么BISS会CRC错误？为什么ABZ会采样不足，AB波形不可信？ GUI要分别有一键启动ABZ、UVW的入口」。前两问是解释题（结论与 RTL 行号依据见 `docs/DEV_STATUS.md` §1.82.1/§1.82.2， 落进 `host/README.md` 的复查记录表），第三问既解释又修，最后一条是新界面需求：

需求	问题	落地位置	验收证据
R13-1 卡片⑨要有"一键启动 ABZ / UVW"入口	UVW 使能位原来只藏在 ⑥倍频倍率卡片折叠的「高级参数」里（8 个复选框 + "应用"），而跑 S4 的"六状态均衡"、看六状态表转正**必须先开 UVW**；卡片⑨ 只显示一行只读的"输出使能 只 ABZ"，用户无从下手	`host/panel_abz.py`：卡片⑨网格新增第 2 行整宽「输出使能」行 = 两个 `setCheckable(True)` 按钮（`btn_abz_en` / `btn_uvw_en`）+ 状态标签（`regs.output_text`，「输出使能：ABZ 使能（0x0109 = 0x0040）」）；新增 `output_requested = Signal(dict)`、`_sync_out_buttons()`（`blockSignals` 拨位）、`restore_output()`（被拒时回滚）；行号后移一行的 `set_max_layout()` 同步改	`selftest_ui.py` 新增离线小节「⑨补2」8 项 + **真机端到端小节**（读 `0x0109` → 点按钮 → bit7 翻转且其它位不变 → 六状态出现合法态且非法态 <5% → 再点复原）⇒ 离线 **115/115 PASS**（原 107）
R13-2 写使能必须"读-改-写"，不能凭猜	`0x0109` 同一个字节里还有 `apol/bpol/zpol/abzdir/uvwpol/uvwdir`，若按 0 或按本地缓存整字节写回，会**把用户设好的极性/方向覆盖掉**；此外串口没连、正忙、还没读回该寄存器时点按钮都不能留假状态	`host/main_window.py`：模块级纯函数 `output_write_value(cur, bits)` = 只在 `decode_output(cur)` 上合并 `abzen/uwven` 再 `(cur & 0xFF00) | encode_output(...)`；`cur is None` 返回 `None`（宁可不写）；`_on_set_output(bits)` 三道闸（未连/忙 ⇒ 提示 + 回滚；没读回 ⇒ 提示"先设备自检" + 回滚；值没变 ⇒ 只记日志），通过后走既有 `task_write_advanced(..., "adv")` 写-回读路径，`_done_adv` 回灌面板	纯函数 5 组断言（含 `0xFF40 + uwven → 0xFFC0` 高字节保留、`None → None`）；无串口时点击 ⇒ 发一次信号但被拒且按钮回滚
R13-3 采样不足的文案数字不能虚报	① `per_cyc = 2.0*self.inter` 多乘 2 ⇒"每箱跨 %.1f 个周期""1024 箱也只有 %.1f 点/周期"**虚报一倍**（判据不受影响，但误导用户）；②旧文案无条件建议"箱数提到 ≥1024"，而 `inter=256` 时 1024 箱也只有 4 点/周期，建议不成立	`host/abz.py`：`per_cyc = float(self.inter)`；"箱数不够"提示按 `need = 8*inter` 与上限 1024 分两支（够得着才让加箱数，够不着就让"同时把 inter 降到 ≤箱数/8"）；`abz.py --selftest` 新增第 19 项（断言"每电周期 256 个 A/B 周期""每箱跨 4.00 个周期"，且不得出现"每箱跨 8.00 个周期"）	`abz.py --selftest` **19/19**；离线全套（`compileall` / `encoder` / `export_png` / `regs` / `ui_prefs` / `alarm` / `recorder` / `selftest_ui`）全绿

需求	问题	落地位置	验收证据
R13-4 ★ 写结果处理器必须按真实返回结构解包（真机验收当场抓出的老 bug）	`host/main_window.py` 的 `_done_adv` / `_done_apply` 写成 `for addr, val, hexs in result["readback"].items()`，而 `task_apply_inter_div` / `task_write_advanced` 的 `readback` 是 `{地址: 值}` **2 元**（3 元的只有 `writes`，原始应答在 `hexes`）⇒ 每次「应用」「一键使能」都在 `_on_task_done` 里抛 `not enough values to unpack (expected 3, got 2)`，被 `except Exception` 吞成一条 fault 日志：**寄存器写进板子了、界面却从不刷新**（`interp.regs_raw` 停在旧值 ⇒ 再点一次算出"已是目标值"、连命令都不发）。`git blame`：来自首次入库 `5e9c326`（2026-10-13），旧自检里 `readback` / `_done_apply` / `_done_adv` 一次都没出现 ⇒ 从没被覆盖	两处改 `for addr, val in result["readback"].items():` + `hexs = (result.get("hexes") or {}).get(addr)`，各留注释；`selftest_ui.py` 新增离线小节「⑨ 补3」2 项，用真机形状的假结果走**完整 `_on_task_done` 分发路径**，断言"没有'结果时出错'日志"且 `interp.regs_raw[0x0109] == 0x00C0`、按钮同步	离线 **117/117 PASS**（115 → 117）；真机 `0x0109 = 0x00C3 → 0x0043 → 0x00C3`（bit7 跟随、同字节其它位不动、复原不残留）；六状态表 `合法 19597 / 非法 0`；记录链路仍 `界面取 61081 帧 = 记录器写 61081 行（差 0）`

仍未做的事：只剩用户拍板的那一条 —— `Ctrl+滚轮` 缩放后时基/角度组合框按"最近档位"显示（显示"满屏 50 ms"而实际 49.15 ms），与"量程："读数不完全一致（用户已明确本轮不做"自定义显示"）。

真机端到端（2026-10-14 晚，用户关掉旧包释放 `COM3` 后跑成）：`python host\selftest_ui.py --real-port COM3 --seconds 6` ⇒ **真机核对 通过 (PASS)**，其中一键使能那小节同时把上面的 R13-4 老 bug 抓了出来（详见 `docs/DEV_STATUS.md` §1.82.6）。

R14 卡片⑨「方波可看 / 还原」一键预设（✅ 2026-10-14 晚）

用户原话（`m11491`）：「为什么ABZ信号波形图不是标准的方波？什么原因导致的？要怎么才能变为标准的方波？」（成因与处方见 `host/README.md` 的复查记录与 `docs/DEV_STATUS.md` §1.83.1）；随后用户直接点单（`m11509`）：「加一个「方波可看」一键预设」。

需求	问题	落地位置	验收证据
R14-1 一键把"每箱跨几个 A/B 周期"配到能看方波	图上画的是**每角度箱里 A/B 为高的平均占比**，一箱跨 `inter/箱数` 个 AB 周期就会把方波平均成阶梯；真机 `inter=256` /256 箱 ⇒ 每箱正好跨 **1.0** 个周期、箱均值恒 ~0.5 ⇒ 曲线在 0.5 附近抖动，既没有 0 也没有 1。要能看方波需 **箱数 ≥ 16×inter**（每周期 ≥16 点），而两处旋钮一个在⑥卡、一个在⑨卡，用户很难自己配	`host/main_window.py`：常量 `SQUARE_POINTS = 16`、模块级纯函数 `square_pair(inter, bins, points=SQUARE_POINTS, choices=None)`（已满足就原样返回；否则优先**只提分箱**、取满足 `b >= it2*points` 的最小档；实在不行才降 inter，且只会降不会升）；`host/panel_abz.py`：卡片⑨ 头部新增 `btn_square`（「方波可看」）/ `btn_square_back`（「还原」）两个按钮 + `square_requested` / `square_restore_requested` 两个信号 + 公开的 `bins()` / `set_bins()`（与「分箱」下拉框同一条 `configure` 路径）；主窗口 `_on_square_preset()`（三道闸：未连/忙、还没读回 `0x0101` / `0x0102`、已满足则只说"不用改"；inter 真变了才写板子）	`square_pair` 5 组断言（`(256,256)→(1024,64)`、`(32,256)→(512,32)`、`(8,256)→(256,8)`、`(2,64)→(64,2)`、`(256,1024)→(1024,64)`）+ 离线 12 项；真机 `分箱 256→1024（目标 1024）`、`inter 256→64（目标 64）`，且 `0x0102` 高字节/`div` 原样保留

需求	问题	落地位置	验收证据
R14-2「还原」必须逐字节写回原值，且不许被"陈旧回读"骗	① `decode→encode` **不是恒等**：`decode_inter_div(0x0000,0x0000)` → (256,1)，`encode_inter_div(256,1)` → `(0x0000,0x0001)` ⇒ 若按解出的 `(inter, div)` 重新编码写回，**原字节复不回去**；②任务内回读若取到陈旧应答（真机实测"写 `0x0101=0x0000` 后回读却是上次的 `0x0040`"），`_done_adv` 会把它灌进 `interp.regs_raw` ⇒ 既污染"现值"、也污染「还原」要用的"原值"	`_on_square_preset()` 在 `_sq_saved` 里额外存**原始寄存器字节** `"raw": {0x0101: …, 0x0102: …}`；`_on_square_restore()` 优先 `task_write_advanced(w, dict(raw))` 原样写回，`raw` 不全时才退回 `task_apply_inter_div(...)`；"已经还原过了"的判据改成比**原始字节**；`task_write_advanced()` 在回读不符时 `time.sleep(0.1)` **再读一次确认**（与 `task_apply_inter_div` 早就有的做法一致）	离线 ⑨补4 断言写入序列就是当初读到的两个原始值（`_FakeW.written`）、无 `raw` 时退回 `apply`、`_FlakyW`（前两次回读给旧值）下 `task_write_advanced` 仍 `ok` 且 `readback == 0x0000`；真机 `按下之前 0x0000 0x0000 → 写回后 0x0000 0x0000`、解出的 `(inter 256, div 1)` 与按下前一致
R14-3 ★ 离线自检必须**真跑**任务函数（真机当场抓出的 `TypeError`）	`_on_square_restore` 里写成 `task_write_advanced(w, dict(raw), "adv")` —— 第三个实参本该给 `task.run` 的 tag ⇒ 任务一执行就 `TypeError: task_write_advanced() takes 2 positional arguments but 3 were given`，被 `_on_task_done` 的兜底 `except` 吞成一条 fault 日志 ⇒ **点「还原」什么都不写**（板上停在预设值）。旧自检用 `task.run` 桩只记 tag、**不调用**任务函数，所以参数错永远显不出来	`host/selftest_ui.py` ⑨补4：`_runf(fn, tag)` 改成真调用并把异常收进 `_errs`；新增检查「任务函数调用参数正确」「回读不一致时会再读一次确认」	离线 **133/133 PASS**（129 → 133）；真机 `[OK] 真机：点「还原」→ 0x0101/0x0102 原样复原（不残留改动）`、`真机核对 通过 (PASS)`

仍未做的事：无（`Ctrl+滚轮` 那条已在 R15 修掉，见下节）。

R15 微调后量程组合框不再"就近显示"（✅ 2026-10-14 晚）

用户原话（`m11938`）：「Ctrl+滚轮 缩放后时基/角度组合框按"最近档位"显示」——这条此前一直挂在 R12/R13 的"待你拍板"里，现在点名要做。

需求	问题	落地位置	验收证据
R15-1 组合框必须显示**真实量程**，不能"就近显示"成相邻档位	`Ctrl+滚轮`（放大态）/ `Shift+←/→` 都是无级微调（`FINE_STEP` 0.98 / 0.9），值不落在档位表里（49.152 ms、359.1°）。`panel_wave.py` / `panel_abz.py` 的 `_sync_controls()` 在 `findData` 失败后走"取最接近档位"的兜底 ⇒ 组合框写「满屏 50 ms」、提示行写「量程：横轴 49.152 ms」，两处对不上	`host/ui_kit.py` 新增 `sync_span_combo(cmb, val, fmt)` + 常量 `CUSTOM_FLAG_ROLE = Qt.UserRole + 1`：命中档位（浮点 1e-9 相对容差）就选该档并**删掉**临时档；是微调值就在「自动」后面插一条临时档（已有则原地更新）；★ 删档后必须**校正下标**（`idx - (1 if cidx < idx else 0)`），否则会选到隔壁档。三处调用：卡片⑧ 时基、卡片⑧ 幅值、卡片⑨ 角度窗口	纯函数五条路径单测（命中/插入/更新/回档/自动）；离线 12 项 + 集成 1 项；离屏截图：放大态 50 ms → `Ctrl+滚轮` ⇒ 组合框 `满屏 49 ms（微调）`、量程行 `横轴 49 ms（手动）`、图标题 `横轴 49 ms` 三处一致且不裁切
R15-2 组合框文本的**位数**要与「量程：」读数一致	第一版用 `%g`（写 49.152）而 `_view_text()` 用 `%.4g`（写 49.15）⇒ 仍然是"两个数字不一样"，自检当场报出来	文本格式与读数对齐：横轴 `满屏 %.4g ms（微调）`、幅值 `±%.2f FS（微调）`、角度 `满屏 %.4g°（微调）`	离线断言"组合框文本与「量程：」读数写同一个数字"（`"49.15" in 文本 and "49.15" in 读数`）
R15-3 回到档位/"恢复自动"时临时档要收干净	临时档若留在列表里，用户以后会看到一个再也用不上的档位；档数不还原也说明状态没收干净	`sync_span_combo()` 在命中档位时先 `removeItem(临时档)`；`reset_auto()` / 选档位都走同一条同步路径	离线断言：回真档位后**档数复原**且 `currentData()` 就是该档（100 ms / 0.50 FS / 270°）；「恢复自动」后档数复原、`currentData() == "auto"`

仍未做的事：无。R12/R13/R14 里挂着的"待拍板"一条（`Ctrl+滚轮` 档位显示）至此关闭。

交付包：2026-10-14 **18:43:02** 重打（第五次），`host\dist_onedir\host_app\host_app.exe` **8,001,154 B**、SHA256 `B6E7F5DA5A9064B66EC726CAA2E34A41FB86DB05B7F9B1BAF7813FBCEE256DF2`、426 文件 / 583,881,293 B。